

JAVA PROGRAMMING MASTERCLASS

輸入與輸出

「讀寫檔案與串流：Java I/O 完整指南」

[← 返回目錄](#)

本章大綱

第一部分：串流基礎

- 22-1 認識串流 (Stream)
- 22-2 InputStream / OutputStream 類別圖

第二部分：Byte I/O

- 22-3 FileInputStream / FileOutputStream
- 22-4 BufferedInputStream / BufferedOutputStream

第三部分：字元 I/O

- 22-5 Writer 和 Reader 類別層次
- 22-6 FileReader / FileWriter
- 22-7 BufferedReader / BufferedWriter

第四部分：系統 I/O

- 22-8 System 類別
- 22-9 PrintStream 類別
- 22-10 Console 類別

第五部分：檔案管理

- 22-11 File 類別

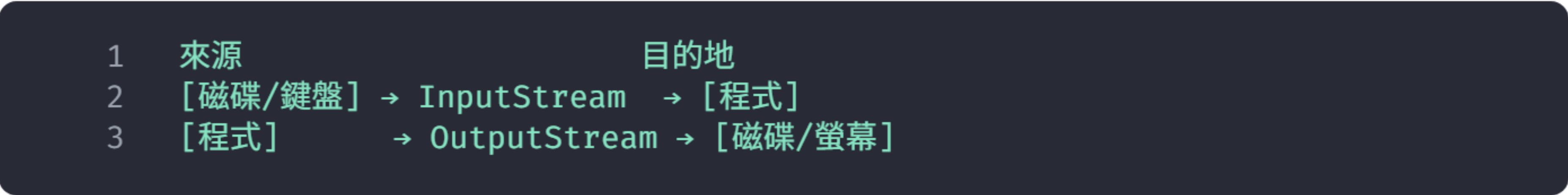
練習與 Q&A

第一部分

串流基礎 (22-1 ~ 22-2)

22-1 認識串流 (Stream)

串流 (Stream) 是資料從來源 (Source) 流向目的地 (Destination) 的抽象通道：



串流的兩大家族：

家族	基底類別	單位	適用場景
Byte Stream	InputStream / OutputStream	8-bit byte	二進位檔 (圖片、音訊)
Char Stream	Reader / Writer	16-bit char	文字檔

💡 原則：文字檔用 Reader/Writer ；二進位檔 (圖片、音訊) 用 InputStream/OutputStream 。

22-2 InputStream / OutputStream 類別圖

InputStream 繼承體系

```
1  InputStream (abstract)
2  |— FileInputStream      ← 讀檔
3  |— ByteArrayInputStream ← 讀 byte[]
4  |— FilterInputStream
5  |   |— BufferedInputStream ← 加緩衝
6  |   |— DataInputStream    ← 讀基本型別
7  |— ObjectInputStream     ← 反序列化
```

OutputStream 繼承體系

```
1  OutputStream (abstract)
2  |— FileOutputStream     ← 寫檔
3  |— ByteArrayOutputStream ← 寫 byte[]
4  |— FilterOutputStream
5  |   |— BufferedOutputStream ← 加緩衝
6  |   |— DataOutputStream    ← 寫基本型別
7  |   |— PrintStream        ← System.out
8  |— ObjectOutputStream   ← 序列化
```

💡 裝飾者模式 (Decorator Pattern) : Java I/O 採用包裝設計，例如 **BufferedInputStream** 包裝 **FileInputStream**，為其加上緩衝功能，不改變原有介面。

InputStream / OutputStream 核心方法

InputStream 方法

說明

```
int read()
```

讀取一個 byte，回傳 0~255；串流結束回傳 **-1**

```
int read(byte[] b)
```

讀入最多 **b.length** 個 byte，回傳實際讀取數

```
int read(byte[] b, int off, int len)
```

從 **off** 位置讀入最多 **len** 個 byte

```
long skip(long n)
```

跳過 **n** 個 byte

```
void close()
```

關閉串流，釋放資源

OutputStream 方法

說明

```
void write(int b)
```

寫出一個 byte

```
void write(byte[] b)
```

寫出整個 byte 陣列

第二部分

Byte I/O (22-3 ~ 22-4)

22-3 FileInputStream – 讀取檔案

最簡單的 byte 讀檔範例：

```
1  try (FileInputStream fis = new FileInputStream("data.bin")) {  
2      int b;  
3      while ((b = fis.read()) != -1) {  
4          System.out.print((char) b);  
5      }  
6  }
```

💡 **try-with-resources**：宣告在 **try(...)** 內的資源會自動呼叫 **close()**，不須手動關閉，避免資源洩漏。

建構子

說明

FileInputStream(String name)

以路徑字串開啟

FileInputStream(File file)

以 File 物件開啟

22-3 FileInputStream – 批次讀取

使用 `byte` 陣列批次讀取，效率更高：

```
1  try (FileInputStream fis = new FileInputStream("photo.jpg")) {  
2      byte[] buffer = new byte[4096];  
3      int bytesRead;  
4      while ((bytesRead = fis.read(buffer)) != -1) {  
5          System.out.println("讀取了 " + bytesRead + " bytes");  
6          // 處理 buffer[0..bytesRead-1]  
7      }  
8  }
```

💡 注意：`read(byte[])` 回傳的是實際讀入的 `byte` 數，不一定等於陣列長度。最後一次讀取可能不足 4096 bytes。

22-3 FileOutputStream – 寫入檔案

建構子

說明

`FileOutputStream(String name)`

覆寫模式 (預設)

`FileOutputStream(String name, boolean append)`

`append=true` 為附加模式

`FileOutputStream(File file)`

以 File 物件開啟

```
1 byte[] data = "Hello, Java I/O!".getBytes();
2 try (FileOutputStream fos = new FileOutputStream("out.txt")) {
3     fos.write(data);
4 }
```

💡 附加模式： `new FileOutputStream("log.txt", true)` 不會清空既有內容，適合寫 log 檔。

22-4 BufferedInputStream / BufferedOutputStream

為什麼需要緩衝？

方式	每次讀寫	系統呼叫次數 (讀 1MB)
無緩衝	1 byte	~1,000,000 次
有緩衝 (8KB)	8192 bytes	~128 次

緩衝區將多次小型 I/O 合併為少量大型 I/O，大幅減少系統呼叫。

```
1  try (BufferedInputStream bis =  
2      new BufferedInputStream(new FileInputStream("big.dat"))) {  
3      byte[] buf = new byte[1024];  
4      int n;  
5      while ((n = bis.read(buf)) != -1) { /* 處理 */ }  
6  }
```

22-4 BufferedOutputStream 範例

```
1  try (BufferedOutputStream bos =  
2      new BufferedOutputStream(new FileOutputStream("out.dat"))) {  
3      for (int i = 0; i < 10000; i++) {  
4          bos.write(i % 256);  
5      }  
6      // close() 會自動 flush()  
7  }
```

建構子

說明

BufferedInputStream(InputStream in)

預設緩衝 8192 bytes

BufferedInputStream(InputStream in, int size)

自訂緩衝大小

BufferedOutputStream(OutputStream out)

預設緩衝 8192 bytes

BufferedOutputStream(OutputStream out, int size)

自訂緩衝大小

第三部分

字元 I/O (22-5 ~ 22-7)

22-5 Reader / Writer 類別層次

Reader 繼承體系

```
1 Reader (abstract)
2 |— InputStreamReader ← byte→char 橋接
3 |   |— FileReader ← 讀文字檔
4 |— BufferedReader ← 加緩衝+readLine
5 |— StringReader ← 讀 String
6 |— CharArrayReader ← 讀 char[]
```

Writer 繼承體系

```
1 Writer (abstract)
2 |— OutputStreamWriter ← char→byte 橋接
3 |   |— FileWriter ← 寫文字檔
4 |— BufferedWriter ← 加緩衝+newLine
5 |— PrintWriter ← println/printf
6 |— StringWriter ← 寫到 String
```

💡 橋接器：**InputStreamReader** / **OutputStreamWriter** 是 byte 串流與 char 串流的橋梁，可指定字元編碼（如 **"UTF-8"**）。

22-6 FileReader — 讀取文字檔

最簡讀取範例：

```
1  try (FileReader fr = new FileReader("hello.txt")) {
2      int ch;
3      while ((ch = fr.read()) != -1) {
4          System.out.print((char) ch);
5      }
6  }
```

建構子

說明

FileReader(String fileName)

以路徑開啟，使用平台預設編碼

FileReader(File file)

以 File 物件開啟

FileReader(String name, Charset cs)

JDK 11+，指定編碼

💡 跨平台編碼：建議使用 `new InputStreamReader(new FileInputStream(file), StandardCharsets.UTF_8)` 明確指定 UTF-8 編碼，避免平台差異。

22-6 FileWriter – 寫入文字檔

建構子

說明

`FileWriter(String fileName)`

覆寫模式

`FileWriter(String fileName, boolean append)`

`true` 為附加模式

`FileWriter(File file, Charset cs)`

JDK 11+，指定編碼

```
1  try (FileWriter fw = new FileWriter("output.txt")) {  
2      fw.write("第一行文字\n");  
3      fw.write("第二行文字\n");  
4  }
```

方法

說明

`write(int c)`

寫出一個字元

`write(String str)`

寫出字串

22-7 BufferedReader – 加緩衝讀取

最重要的功能：逐行讀取 `readLine()`

```
1  try (BufferedReader br =  
2      new BufferedReader(new FileReader("data.csv"))) {  
3      String line;  
4      while ((line = br.readLine()) != null) {  
5          System.out.println(line);  
6      }  
7  }
```

💡 注意：`readLine()` 回傳 `null` (不是 -1) 表示檔案結束；回傳值不含行尾的換行符號。

22-7 BufferedWriter – 加緩衝寫入

```
1  try (BufferedWriter bw =  
2      new BufferedWriter(new FileWriter("report.txt"))) {  
3      bw.write("姓名\t成績");  
4      bw.newLine();           // 平台相容的換行  
5      bw.write("Alice\t95");  
6      bw.newLine();  
7  }
```

方法

說明

`write(String s)`

寫出字串

`write(String s, int off, int len)`

寫出字串的一部分

`newLine()`

寫出平台換行符 (Windows `\r\n` , Unix `\n`)

`flush()`

強制將緩衝區送出

字元 I/O 完整流程圖

```
1  文字檔 (UTF-8)
2      ↓ FileInputStream
3      ↓ InputStreamReader(StandardCharsets.UTF_8)  ← byte → char
4      ↓ BufferedReader                               ← 加緩衝 + readLine
()
5      [Java 程式邏輯]
6      ↓ BufferedWriter                               ← 加緩衝 + newLine()
7      ↓ OutputStreamWriter(StandardCharsets.UTF_8) ← char → byte
8      ↓ FileOutputStream
9  文字檔 (UTF-8)
```

💡 最佳實踐：明確指定編碼：

```
new BufferedReader(new InputStreamReader(new FileInputStream("f.txt"), StandardCharsets.UTF_8))
```

第四部分

系統 I/O (22-8 ~ 22-10)

22-8 System 類別 – 標準串流

Java 啟動時自動建立三個靜態串流欄位：

欄位	類型	預設目的地	用途
<code>System.in</code>	<code>InputStream</code>	鍵盤輸入	讀取使用者輸入
<code>System.out</code>	<code>PrintStream</code>	標準輸出 (螢幕)	正常訊息輸出
<code>System.err</code>	<code>PrintStream</code>	標準錯誤 (螢幕)	錯誤訊息輸出

```
1 // 讀取鍵盤輸入 (包裝成 BufferedReader 使用)
2 BufferedReader br = new BufferedReader(
3     new InputStreamReader(System.in));
4 String line = br.readLine();
```

22-9 PrintStream 類別

`System.out` 和 `System.err` 的實際型別，提供方便的格式化輸出。

方法	說明
<code>print(x)</code>	輸出任意型別，不換行
<code>println(x)</code>	輸出後加換行
<code>printf(format, args ...)</code>	格式化輸出 (同 C 的 <code>printf</code>)
<code>format(format, args ...)</code>	與 <code>printf</code> 等效

```
1 System.out.printf("%-10s %5d %8.2f%n", "Alice", 95, 98.75);
2 System.out.printf("%-10s %5d %8.2f%n", "Bob", 82, 80.50);
3 // Alice          95      98.75
4 // Bob            82      80.50
```


22-9 printf 常用格式符號

格式符	說明	範例	輸出
<code>%d</code>	十進位整數	<code>printf("%d", 42)</code>	<code>42</code>
<code>%5d</code>	寬度 5，靠右	<code>printf("%5d", 42)</code>	<code>42</code>
<code>%-5d</code>	寬度 5，靠左	<code>printf("%-5d", 42)</code>	<code>42</code>
<code>%f</code>	浮點數（預設 6 位小數）	<code>printf("%f", 3.14)</code>	<code>3.140000</code>
<code>%.2f</code>	2 位小數	<code>printf("%.2f", 3.14)</code>	<code>3.14</code>
<code>%s</code>	字串	<code>printf("%s", "Hi")</code>	<code>Hi</code>
<code>%n</code>	平台換行符	—	換行
<code>%b</code>	布林值	<code>printf("%b", true)</code>	<code>true</code>

22-10 Console 類別

`System.console()` 回傳與 JVM 關聯的 `Console` 物件（若無終端機則回傳 `null`）：

```
1  Console console = System.console();
2  if (console == null) {
3      System.err.println("沒有可用的終端機");
4      return;
5  }
6  String username = console.readLine("請輸入帳號：");
7  char[] password = console.readPassword("請輸入密碼：");
8  System.out.printf("歡迎，%s！%n", username);
9  java.util.Arrays.fill(password, ' '); // 安全清除密碼
```

方法

說明

`readLine(String fmt, ...)`

先顯示提示，再讀一行

`readPassword(String fmt, ...)`

讀密碼（不回顯），回傳 `char[]`

`format(String fmt, ...)`

格式化輸出到終端

第五部分

檔案管理（22-11 File 類別）

22-11 File 類別 – 基本概念

java.io.File 代表一個路徑（可能是檔案或目錄），本身不做 I/O 操作：

```
1 File f1 = new File("data.txt");           // 相對路徑
2 File f2 = new File("/home/user/data.txt"); // 絕對路徑
3 File f3 = new File("/home/user", "data.txt"); // 目錄 + 檔名
4 File dir = new File("reports");
5 File f4 = new File(dir, "2024.csv");        // File + 檔名
```

建構子

說明

File(String pathname)

路徑字串

File(String parent, String child)

父路徑 + 子名稱

File(File parent, String child)

父 File + 子名稱

💡 注意：建立 **File** 物件不會建立實際檔案，只是表示一個路徑。

22-11 File 類別 – 查詢方法

方法	回傳	說明
<code>exists()</code>	<code>boolean</code>	路徑是否存在
<code>isFile()</code>	<code>boolean</code>	是否為一般檔案
<code>isDirectory()</code>	<code>boolean</code>	是否為目錄
<code>canRead()</code> / <code>canWrite()</code>	<code>boolean</code>	是否可讀/可寫
<code>getName()</code>	<code>String</code>	取得檔名 (含副檔名)
<code>getPath()</code>	<code>String</code>	取得路徑字串
<code>getAbsolutePath()</code>	<code>String</code>	取得絕對路徑
<code>length()</code>	<code>long</code>	檔案大小 (bytes)

22-11 File 類別 – 建立與刪除

```
1  File dir = new File("output/reports");
2  if (!dir.exists()) {
3      boolean ok = dir.mkdirs(); // 連同父目錄一起建立
4      System.out.println("建立目錄：" + ok);
5  }
6
7  File f = new File(dir, "result.txt");
8  if (f.createNewFile()) { // 建立空檔案 (已存在則 false)
9      System.out.println("建立檔案：" + f.getAbsolutePath());
10 }
11
12 f.delete(); // 刪除檔案
13 dir.delete(); // 刪除空目錄
```

方法

說明

mkdir()

建立單一目錄 (父目錄須存在)

mkdirs()

建立目錄及所有必要的父目錄

createNewFile()

建立空檔案 已存在回傳

false

22-11 File 類別 – 列舉目錄

```
1  File dir = new File("src");
2  if (dir.isDirectory()) {
3      File[] files = dir.listFiles();
4      if (files != null) {
5          for (File f : files) {
6              String type = f.isDirectory() ? "[目錄]" : "[檔案]";
7              System.out.printf("%s %-30s %8d bytes%n",
8                              type, f.getName(), f.length());
9          }
10     }
11 }
```

方法	回傳	說明
<code>list()</code>	<code>String[]</code>	子項目名稱陣列
<code>listFiles()</code>	<code>File[]</code>	子項目 File 物件陣列
<code>list(FilenameFilter)</code>	<code>String[]</code>	依過濾條件篩選名稱

22-11 FilenameFilter 過濾

使用 lambda 過濾特定副檔名的檔案：

```
1  File dir = new File(".");
2  File[] javaFiles = dir.listFiles(
3      (d, name) → name.endsWith(".java")
4  );
5
6  if (javaFiles != null) {
7      for (File f : javaFiles) {
8          System.out.println(f.getName() + " - " + f.length() + " byte
9      );
10     }
```

💡 現代替代方案：JDK 7+ 提供 `java.nio.file.Files` 與 `Path`，功能更強大（支援 walk、遞迴列舉等），是新專案的推薦選擇。

練習一：Byte 檔案複製

任務說明

撰寫一個程式，將指定的二進位檔案（如圖片）複製到另一個路徑。

需求：

1. 使用 `FileInputStream` 和 `FileOutputStream` 實作
2. 加入 `BufferedInputStream` / `BufferedOutputStream` 提升效能
3. 使用 `try-with-resources` 確保資源釋放
4. 顯示複製的 byte 數量

預期輸出：

```
1 複製完成：photo.jpg → photo_backup.jpg
2 共複製 204800 bytes
```

練習一：解題提示

```
1  long total = 0;
2  try (BufferedInputStream bis = new BufferedInputStream(
3      new FileInputStream("photo.jpg"));
4      BufferedOutputStream bos = new BufferedOutputStream(
5      new FileOutputStream("photo_backup.jpg"))) {
6      byte[] buf = new byte[8192];
7      int n;
8      while ((n = bis.read(buf)) != -1) {
9          bos.write(buf, 0, n);
10         total += n;
11     }
12 }
13 System.out.println("共複製 " + total + " bytes");
```

關鍵點：

- 多個 AutoCloseable 資源可在同一個 try-with-resources 中宣告，用分號分隔
- 關閉順序與宣告順序相反（先關 bos 再關 bis）
- **bos.write(buf, 0, n)** 而非 **bos.write(buf)**，只寫實際讀到的 bytes

練習二：逐行讀取文字檔

任務說明

撰寫一個程式，讀取 CSV 文字檔，計算每位學生的平均成績並輸出報表。

輸入檔案 `scores.csv`：

```
1 Alice,85,90,78
2 Bob,70,88,95
3 Carol,92,76,84
```

預期輸出：

```
1 Alice : 84.33
2 Bob : 84.33
3 Carol : 84.00
```

需求：使用 `BufferedReader` + `FileReader`，`readLine()` 逐行讀取，`split(",")` 切割欄位，`try-with-resources` 管理資源。

練習二：解題提示

```
1  try (BufferedReader br =  
2      new BufferedReader(new FileReader("scores.csv"))) {  
3      String line;  
4      while ((line = br.readLine()) != null) {  
5          String[] parts = line.split(",");  
6          String name = parts[0];  
7          double sum = 0;  
8          for (int i = 1; i < parts.length; i++)  
9              sum += Integer.parseInt(parts[i]);  
10         System.out.printf("%-8s: %.2f%n",  
11             name, sum / (parts.length - 1));  
12     }  
13 }
```

關鍵步驟：

1. `readLine()` + `split(",")` 是 Java 文字處理最常見的組合
2. `parts.length - 1` 是分數欄位的數量 (排除姓名欄)
3. `Integer.parseInt()` 將字串轉為 int

練習三：目錄瀏覽工具

任務說明

撰寫一個方法，遞迴列出指定目錄下所有的檔案與子目錄，以樹狀結構輸出：

```
1  [目錄] src
2    [目錄] com/example
3      [檔案] Main.java (1234 bytes)
4      [檔案] Utils.java (567 bytes)
5      [檔案] module-info.java (89 bytes)
```

需求：

1. 使用 `File.isDirectory()` 判斷是否為目錄
2. 使用 `File.listFiles()` 取得子項目（注意 null 檢查）
3. 遞迴方法遍歷子目錄
4. 用縮排（indent 參數）表示目錄層次

練習三：解題提示

```
1  void printTree(File dir, String indent) {
2      File[] files = dir.listFiles();
3      if (files == null) return; // null 檢查
4      for (File f : files) {
5          if (f.isDirectory()) {
6              System.out.println(indent + "[目錄] " + f.getName());
7              printTree(f, indent + "  ");
8          } else {
9              System.out.printf("%s[檔案] %s (%d bytes)%n",
10                  indent, f.getName(), f.length());
11          }
12      }
13  }
14  // 呼叫：printTree(new File("src"), "");
```

重點：

- `listFiles()` 可能回傳 `null` (目錄不存在或 I/O 錯誤)，需先檢查
- 遞迴時 indent 加兩個空格表示深一層
- 若需排序可呼叫 `Arrays.sort(files)`

Q & A

課程結束

Java I/O：byte 串流、字元串流、系統 I/O、File 類別